

СИСТЕМА АВТОМАТИЧЕСКОЙ БЕСШОВНОЙ РОТАЦИИ КРИПТОГРАФИЧЕСКИХ КЛЮЧЕЙ АВТОРИЗАЦИИ В ВЫСОКОНАГРУЖЕННОЙ МИКРОСЕРВИСНОЙ ПЛАТФОРМЕ

Одной из частых операций в современных .NET приложениях [1] является поиск типов в сборке, удовлетворяющих предикату, который программист задает в своем коде. Такой подход обеспечивает хорошую расширяемость приложения, убирает избыточный код и облегчает разработку, снижая когнитивную нагрузку на программиста. Для реализации такого функционала используется стандартный механизм рефлексии .NET [2], однако он создает некоторые ограничения [3]:

1. Ошибки времени исполнения при сборке в виде нативных бинарных файлов.
2. Выделение динамической памяти.
3. Ухудшение пользовательского опыта разработчика.

Цель работы – разработать инструмент, позволяющий обойти все вышеперечисленные ограничения, обеспечивая при этом удобный и интуитивный интерфейс для программиста. Для достижения этой цели необходимо решение следующих *задач*:

1. Обзор существующих реализаций замены рефлексии в .NET.
2. Проведение сравнительного анализа найденных вариантов.
3. Предложение унифицированного инструмента оптимизации, покрывающего задачи, представленные в существующих подходах.
4. Реализация инструмента с автоматизацией тестирования и публикации в репозитории пакетов.
5. Демонстрация оптимизации на основе бенчмарков [4].

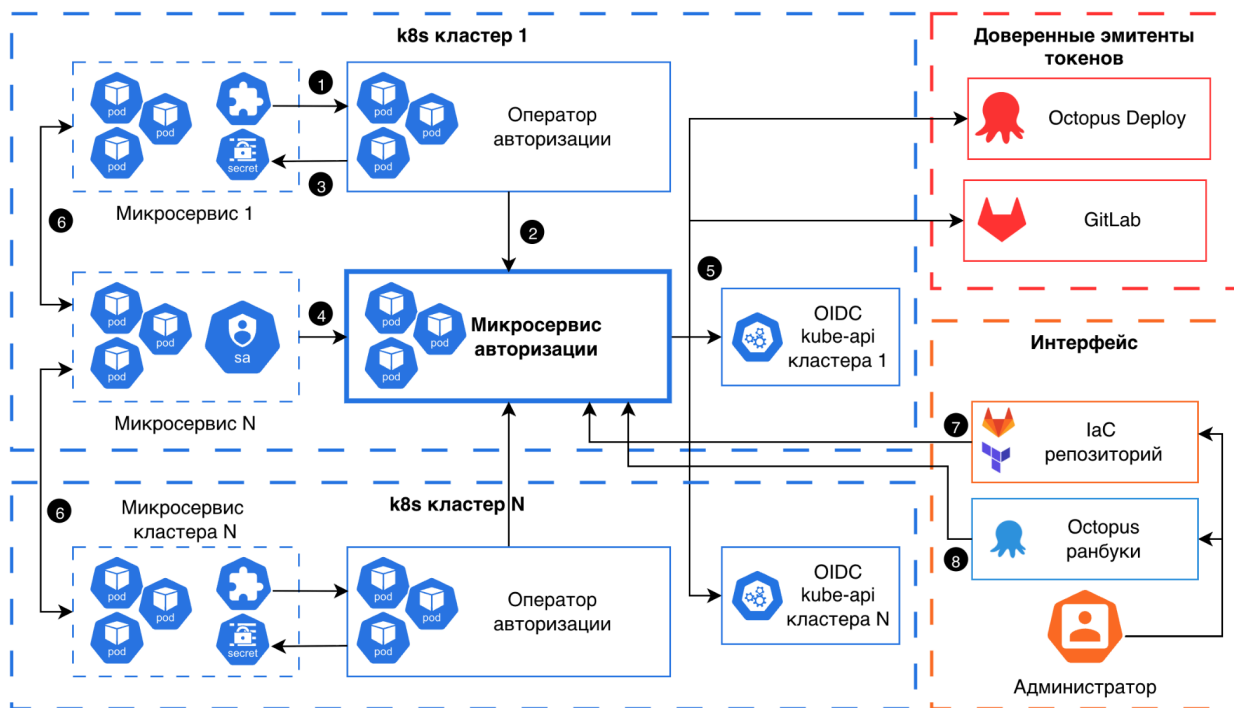
Для *реализации* используется Roslyn Compiler Platform [5]. Это достаточно новый SDK для разработчиков, позволяющий встраиваться в процесс компиляции .NET сборки. Начиная с .NET 6 (2021 год) Microsoft представила инструмент инкрементальных генераторов кода [6], который базируется на этой платформе. Генераторы на основе анализа предоставленных синтаксических деревьев могут создавать файлы с кодом, добавляя их к компиляции сборки. Таким образом процесс сбора информации о типах можно перенести из стадии времени выполнения программы на стадию времени компиляции.

Архитектура разработанного инкрементального генератора изображена на рис. 1. Разработчик создает метод, помечает его атрибут AotAssemblyScan и атрибутами фильтрации, а метод возвращающий список подходящих типов. Далее хост (IDE или компилятор) вызывает код в AssemblyScanGenerator, который получает синтаксические деревья, фильтрует типы, используя критерии, определенные разработчиком, генерирует код и кэширует результат. Критерии фильтрации также определяются на основе атрибутов, а за их обработку отвечают наследники ITypeCriteria.

Автоматическое тестирование генерируемого кода использует подход snapshot тестирования на основе библиотек xUnit и Verify.SourceGenerators. Запуск тестов происходит автоматически в GitHub при создании запроса на слияние кода.

Публикация пакета на NuGet и GitHub Container Registry происходит автоматически при создании тега с номером версии на main ветке.

Результат был проверен на реальном коммерческом приложении из 40 тысяч строк. На рис. 2 можно увидеть прирост по времени в 4967 раз и уменьшение количества затраченной памяти в 142 раза.



- ❶ Оператор раз в 30 секунд читает CR – спецификацию запрашиваемых токенов из микросервиса авторизации.
- ❷ Оператор на основе спецификации получает токены и публичные ключи у микросервиса авторизации.
- ❸ Оператор записывает полученные данные авторизации в секрет микросервиса.
- ❹ Микросервис получает токены от ервиса авторизации, если данные известны только в рантайме. Авторизация через SA токен.
- ❺ Для авторизации субъектов и возможностей федерации сервис авторизации запрашивает публ. ключи доверенных систем.
- ❻ Микросервисы отправляют и авторизуют запросы с помощью полученных данных. В том числе, между разными кластерами.
- ❼ Администратор управляет конфигурацией микросервиса авторизации (клиенты, политики, эмитенты) через IaC репозиторий.
- ❽ Octopus ранбуки дают доступ к ручному администрированию: принудительный запуск ротации, выпуск токенов для отладки.

Рисунок 1 – Архитектура решения

Таким образом, был разработана библиотека, которая в декларативном стиле позволяет находить типы в сборке, обеспечивая намного большую производительность, чем стандартный подход на основе рефлексии.

ЛИТЕРАТУРА

1. .NET. [Электронный ресурс] Режим доступа: <https://dotnet.microsoft.com/en-us/>
2. Slimák Ondřej Source Generators in C#: дис. канд. инф. наук: Brno, 2022. - 77 с.
3. Stanislav Luke's API for C# Code Generation: дис. канд. инф. наук: Prague, 2020. - 55 с.
4. Akinshin. A Pro .NET Benchmarking: The Art of Performance Measurement. - 1-е изд. - New York: Apress, 2019. - 687 с.
5. Roslyn Compiler Platform. [Электронный ресурс] Режим доступа: <https://learn.microsoft.com/en-us/dotnet/csharp/roslyn-sdk/>
6. Incremental Source Generators. [Электронный ресурс] Режим доступа: <https://github.com/dotnet/roslyn/blob/main/docs/features/incremental-generators.md>